

Computer Vision

Assignment 2

Submitted to:

Dr. Adnan Shah

Submitted by:

Quratulain Malik

22-CS-103

Computer Vision Assignment 2: Application of LBP and LTP on an Image

1. Input Image and Preprocessing

The solution begins by loading the input image and converting it to grayscale⁶, which is the standard format for texture analysis.

1.1 Original Grayscale Image



1.2 Python Code Snippet: Loading and Grayscale

Python

```
import cv2
```

```
import numpy as np
```

```
# ... (rest of imports)
```

```
# Load image (assuming original_image_color is the loaded image)
```

```
original_image_gray = cv2.cvtColor(original_image_color, cv2.COLOR_BGR2GRAY) # Convert to grayscale
```

2. Feature Extraction and Patterns

2.1 Local Binary Pattern (LBP) Implementation

LBP extracts features by comparing each center pixel to its 8 neighbors⁷.

LBP Pattern Output



Python Code Snippet: LBP Core Logic

Python

```
def calculate_lbp(image):  
    # ... (initialization)  
  
    for i in range(1, rows - 1):  
        for j in range(1, cols - 1):  
            center = image[i, j]  
            lbp_value = 0  
  
            # Example for one neighbor (Top-Right):
```

```

if image[i - 1, j + 1] >= center:
    lbp_value += 2**0
# ... (Repeat for 7 other neighbors)

LBP_pattern[i, j] = lbp_value
return LBP_pattern

```

Execution

```
lbp_pattern = calculate_lbp(original_image_gray)
```

2.2 Local Ternary Pattern (LTP) Implementation

LTP extends LBP using a threshold τ to introduce a 'dead zone'⁹. This results in two patterns: Upper (for neighbors significantly brighter than center) and Lower (for neighbors significantly darker). Here, $\tau=5$ was used.

LTP Pattern Outputs



Python Code Snippet: LTP Core Logic

Python

```
def calculate_ltp(image, threshold=5):
```

```

# ... (initialization)

for i in range(1, rows - 1):
    for j in range(1, cols - 1):
        center = image[i, j]
        upper_value = 0
        lower_value = 0

        # Iterate through 8 neighbors:
        for k, (dr, dc) in enumerate(offsets):
            neighbor = image[i + dr, j + dc]
            power_of_2 = 2**k

            # Upper Pattern (Significantly brighter)
            if neighbor > center + threshold:
                upper_value += power_of_2

            # Lower Pattern (Significantly darker)
            elif neighbor < center - threshold:
                lower_value += power_of_2

        LTP_upper_pattern[i, j] = upper_value
        LTP_lower_pattern[i, j] = lower_value
    return LTP_upper_pattern, LTP_lower_pattern

```

Execution

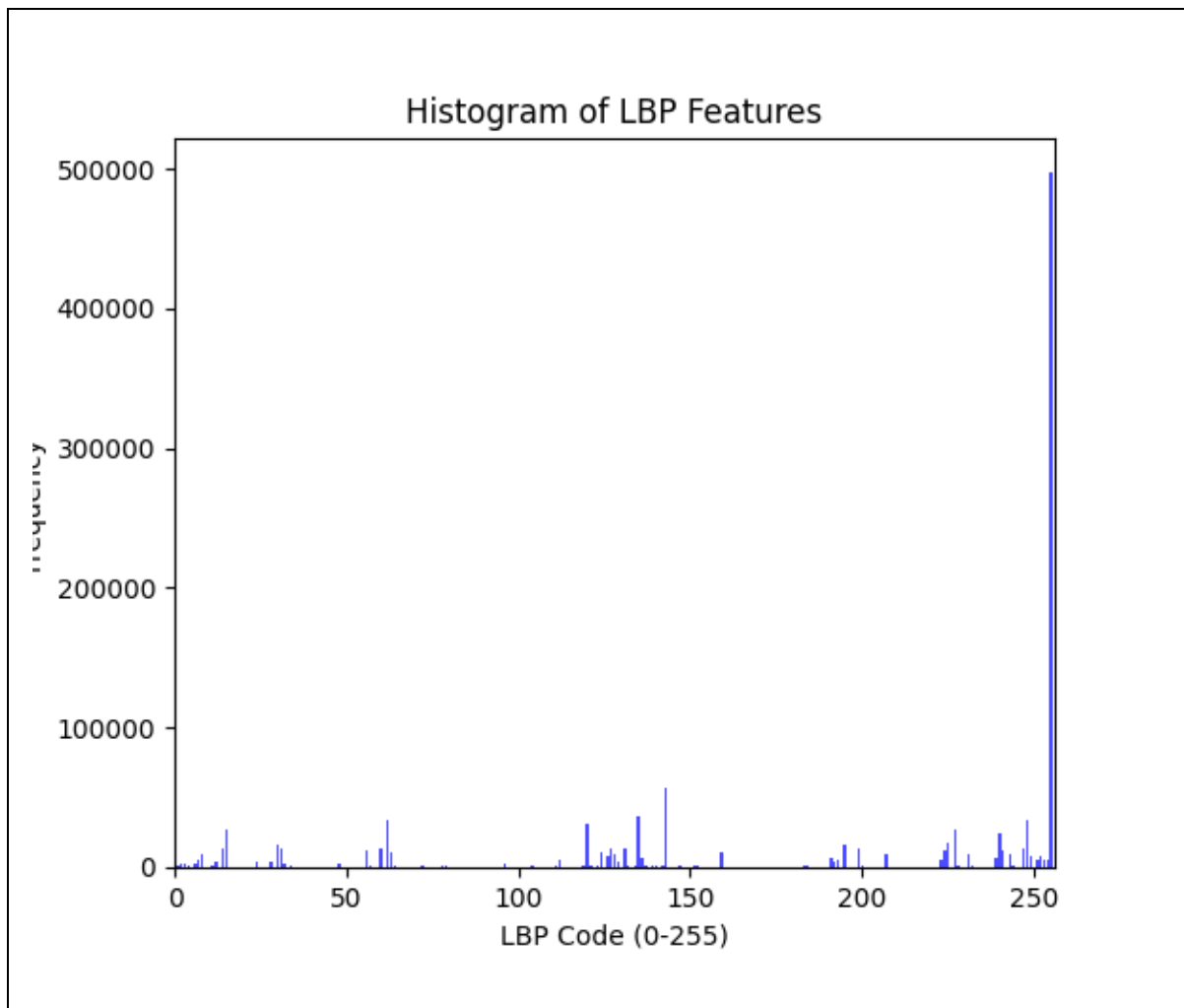
```
LTP_THRESHOLD = 5
```

```
ltp_upper_pattern, ltp_lower_pattern = calculate_ltp(original_image_gray,  
threshold=LTP_THRESHOLD)
```

3. Histogram Plotting and Feature Vectors

The histograms represent the frequency count of each feature code, forming the final feature vectors for comparison¹¹.

3.1 Histogram of LBP Features



Python Code Snippet: LBP Histogram Calculation and Plotting

Python

```

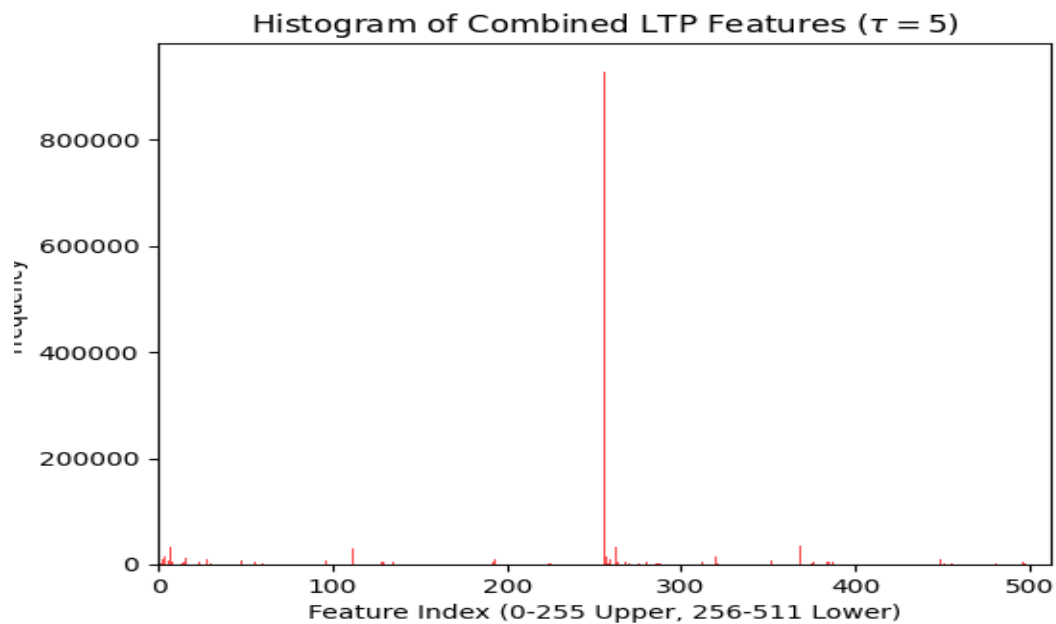
import matplotlib.pyplot as plt

# Calculate histogram (256 bins)
lbp_hist, _ = np.histogram(lbp_pattern.ravel(), bins=np.arange(0, 257), range=(0, 256))

plt.figure(figsize=(6, 5))
plt.bar(np.arange(256), lbp_hist, color='blue', alpha=0.7)
plt.title('Histogram of LBP Features')
plt.xlabel('LBP Code (0-255)')
plt.ylabel('Frequency')
plt.show() # Display the plot

```

3.2 Histogram of Combined LTP Features



Python Code Snippet: LTP Histogram Calculation and Plotting

Python

```
# Calculate Upper and Lower histograms (256 bins each)
```

```
ltp_upper_hist, _ = np.histogram(ltp_upper_pattern.ravel(), bins=np.arange(0, 257), range=(0, 256))
```

```
ltp_lower_hist, _ = np.histogram(ltp_lower_pattern.ravel(), bins=np.arange(0, 257), range=(0, 256))
```

```
# Concatenate for the 512-dimension feature vector
```

```
ltp_feature_vector = np.concatenate((ltp_upper_hist, ltp_lower_hist))
```

```
plt.figure(figsize=(6, 5))
```

```
plt.bar(np.arange(512), ltp_feature_vector, color='red', alpha=0.7)
```

```
plt.title(f'Histogram of Combined LTP Features ( $\tau$ ={LTP_THRESHOLD})')
```

```
plt.xlabel('Feature Index (0-255 Upper, 256-511 Lower)')
```

```
plt.ylabel('Frequency')
```

```
plt.show() # Display the plot
```

4. Comparison and Threshold Discussion

Comparison of LBP and LTP Results ¹²

LBP, a binary system, is highly sensitive to subtle intensity changes, resulting in a **dense pattern** and a **distributed histogram** across its 256 bins. LTP, a ternary system, introduces a 'dead zone' defined by the threshold τ , which results in **sparser LTP Patterns** that primarily capture strong edges. This suppression of subtle variations causes the **LTP histogram to be dominated by the zero-value bins** (representing the 'dead zone'), making the LTP feature vector less susceptible to local noise but twice the length (512 bins) to preserve sign information.

Discussion on how Thresholds Affect Texture Representation ¹³

The threshold τ in LTP is crucial for controlling its robustness.

- A small τ (approaching 0) causes LTP to function almost like LBP, making it susceptible to noise but capturing finer detail.
- A large τ (e.g., 8-15) widens the 'dead zone,' increasing **robustness to noise** but causing the **loss of fine texture details**. Only high-contrast features survive the ternary comparison. Choosing an optimal τ (like the $\tau=5$ used here) balances noise suppression with texture information retention.